# Examen partiel – 3 novembre 2026

**durée : 2 heures**

Pour chacun des algorithmes, on justifiera avec soin :

- que l'algorithme termine.
- que l'algorithme renvoie bien le résultat demandé.

Pour les boucles « pour », on adoptera les conventions suivantes. Si $a, b \in \mathbb{Z}$, « pour $i$ allant de $a$ à $b$ » signifie « pour $i$ parcourant en croissant l'intervalle $[\![a, b]\!]$ ». Lorsque $b < a$, cet intervalle est vide, donc aucune des instructions dans la boucle « pour » n'est effectuée (ce sera par exemple le cas si on écrit « pour $i$ allant de 1 à $n$ », avec $n = 0$). On peut aussi écrire les algorithmes en Python, dans ce cas le code Python doit être correct, en particulier l'utilisation de « range » doit être correcte. On rappelle qu'en Python, les listes sont numérotées à partir de zéro, `len(L)` retourne la longueur d'une liste, et que `L[a:b]` retourne la liste extraite (éventuellement vide) obtenue en prenant les indices entre $a$ inclus et $b$ exclus. L'expression `L[a:]` retourne la liste (éventuellement vide) extraite en prenant les indices à partir de $a$ (inclus) et l'expression `L[:b]` retourne la liste (éventuellement vide) extraite en prenant les indices inférieurs à $b$ (exclus). La concaténation de deux listes se fait avec l'opérateur `+`. Pour $a$, $b$ et $c$ entiers naturels et $b$ et $c$ non nuls, on peut utiliser que `(a//b)//c = a//(bc)`.

(Rappel : en cas de blocage, la méthodologie attendue est d'abord de faire de nombreux exemples.)

**Exercice 1.** *On considère la fonction suivante, qui prend deux entiers relatifs en entrée. Termine-t-elle ? Que fait-elle ? Préciser les définitions si besoin.*

```
def f(n,k):
  if k < 0 or k > n:
    return 0
  if k == 0 and n == 0:
    return 1
  return f(n-1,k)+f(n-1,k-1)
```

**Exercice 2.** *On considère la fonction suivante, qui prend deux entiers naturels en entrée. Montrer qu'elle renvoie le produit de a par b. Cet algorithme porte le nom de « multiplication russe ».*

```
def f(a,b):
    resultat = 0

    while a > 0:
        if a % 2 == 1:
            resultat = resultat + b

        a = a // 2
        b = b * 2

    return resultat
```

**Exercice 3.** *Écrire une version récursive du même algorithme. Prouver la terminaison et correction.*

**Exercice 4.** *L'algorithme suivant prend en entrée deux entiers naturels non tous deux nuls. Termine-t-il ? Que fait-il ?*

```
def f(a,b):
    while b != 0:
        a, b = b, a % b
    return a
```

*Écrire une version récursive de cet algorithme, sans prouver la terminaison ni la correction.*

**Exercice 5.** *La fonction suivante prend en entrée un nombre $x$ et une liste $L$ de nombres qui est déjà triée par ordre croissant. Termine-t-elle ? Que fait-elle ?*

```
def f(x,L):
  if L == [] : return False
  p = len(L)//2
  if L[p] == x : return True
  if L[p] < x  : return f(x,L[p+1:])
  if L[p] > x  : return f(x,L[:p])
```

**Exercice 6.** *Soient $L$ et $L'$ deux listes* ***non vides*** *et $n$ un entier naturel. On dit que « $L$ est une sous-liste de $L'$ en position $n$ » si : $\forall k \in [\![0, long(L)-1]\!]$, $L'[n+k]$ est défini et égal à $L[k]$. De plus, on dit que « $L$ est une sous-liste de $L'$ » s'il existe un entier naturel $n$ tel que $L$ soit une sous-liste de $L'$ en position $n$.*

*Par exemple, $[2,1,7]$ est une sous-liste de $[2,5,0,2,1,7,3,2]$.*

*Écrire une fonction* `estSousListe(L,M)` *prenant en entrée deux listes $L$ et $M$ non vides et renvoyant le booléen vrai ou faux suivant si $L$ est une sous-liste de $M$.*

*On pourra commencer par écrire une fonction* `estSousListePosition(L,M,n)` *prenant deux listes non vides et un entier naturel $n$ et renvoyant le booléen vrai ou faux suivant si $L$ est une sous-liste de $M$ en position $n$.*

# Proposition de corrigé

## Exercice 1 : coefficients binomiaux

Dans cette correction, on suppose que les coefficients binomiaux sont définis seulement pour $0 \leq k \leq n$. Avec ce choix, la formule de Pascal est :

$$\forall n \in \mathbb{N}^*, \forall k \in [\![1, n]\!], \binom{n+1}{k} = \binom{n}{k} + \binom{n}{k-1}.$$

Montrons que la fonction renvoie le coefficient binomial si $0 \leq k \leq n$, et zéro sinon.

Preuve : si $n < 0$, la fonction renvoie zéro tout de suite avec le test conditionnel.

Pour les $n$ positifs, on procède par récurrence sur $n$. On note $\mathcal{A}(n)$ l'assertion « pour tout $k$ relatif, l'appel à $f(n, k)$ termine et renvoie $\binom{n}{k}$ si $0 \leq k \leq n$ et zéro sinon. »

- L'assertion $\mathcal{A}(0)$ est vraie. Soit $k$ relatif. Si $k = 0$ on renvoie $1 = \binom{0}{0}$, sinon zéro.
- Soit $n \in \mathbb{N}$. Supposons $\mathcal{A}(n)$. Montrons $\mathcal{A}(n+1)$. Soit $k \in \mathbb{Z}$. Il y a plusieurs cas :
  - Si $k < 0$ ou $k > n + 1$ on renvoie bien zéro tout de suite.
  - Si $k = 0$, alors $f(n, 0)$ termine et renvoie $\binom{n}{0} = 1$ par hypothèse de récurrence, et toujours par hypothèse de récurrence $f(n, -1)$ termine et renvoie zéro. Finalement on renvoie la somme de ces deux valeurs donc 1 ce qui est bien la bonne valeur.
  - Si $1 \leq k \leq n$ (ceci n'a donc lieu que si $n \geq 1$) : l'appel à $f(n + 1 - 1, k)$ termine et renvoie $\binom{n}{k}$ car $1 \leq k \leq n$ (et donc $0 \leq k \leq n$), celui à $f(n + 1 - 1, k - 1)$ termine et renvoie $\binom{n}{k-1}$ car $0 \leq k - 1 \leq n - 1$ (et donc $0 \leq k - 1 \leq n$). La fonction renvoie donc $\binom{n}{k} + \binom{n}{k-1}$ ce qui est égal à $\binom{n+1}{k}$ par la formule de Pascal.
  - Si $k = n+1$, alors $f(n, k) = f(n, n+1)$ termine et renvoie zéro par hypothèse de récurrence, et toujours par hypothèse de récurrence $f(n, k-1) = f(n, n)$ termine et renvoie $\binom{n}{n} = 1$. Finalement on renvoie la somme de ces deux valeurs, donc 1 ce qui est bien la bonne valeur.

## Exercice 2 : multiplication russe, terminaison et correction

(Note : au fond, l'algorithme calcule l'écriture en base 2 de $a = \sum \epsilon_k 2^k$. Il renvoie $\sum \epsilon_k b 2^k = ba$ en calculant cette somme en sommant les termes un par un.)

**Terminaison** : tant que la boucle tourne, les valeurs de prises par la variable « `a` » forment une suite d'entiers naturels strictement décroissante. Comme il n'existe pas de suite infinie strictement décroissante d'entiers naturels, la boucle tourne donc un nombre fini de fois.

**Correction** : On note $K$ le nombre de tours de boucle effectués ($K = 0$ si on ne rentre pas dans la boucle du tout.) Pour $k \in [\![0, K]\!]$, notons $a_k$, $b_k$ et $r_k$ les valeurs des variables `a`, `b` et `resultat` à la fin du $k$-ème tour de boucle, ou avant la boucle pour $k = 0$. Notons enfin $\mathcal{A}(k)$ l'assertion :

$$\begin{cases} a_k &= a_0 // 2^k \\ b_k &= b_0 2^k \\ r_k &= b_0 \times (a_0 \% 2^k) \end{cases}$$

Montrons par récurrence : $\forall k \in [\![0, K]\!], \mathcal{A}(k)$.

- initialisation : ok.
- Soit $k \in [\![0, K-1]\!]$. Supposons $\mathcal{A}(k)$. Montrons alors $\mathcal{A}(k+1)$.

  La première instruction conditionnelle donne : $r_{k+1} = r_k + b_k(a_k \% 2)$. Ensuite, en utilisant l'hypothèse de récurrence :

$$\begin{aligned} r_{k+1} &= r_k + b_k(a_k\%2) \\ &= r_k + b_k\left(a_k - 2 \times a_k//2\right) \\ &= b_0(a_0\%2^k) + b_0 2^k\left(a_0//2^k - 2 \times a_0//2^{k+1}\right) \\ &= b_0\left[a_0\%2^k + 2^k \times a_0//2^k - 2^{k+1} \times a_0//2^{k+1}\right] \\ &= b_0\left[a_0 - 2^{k+1} \times a_0//2^{k+1}\right] \\ &= b_0\left[a_0\%2^{k+1}\right] \end{aligned}$$

(Rédaction sans doute sous-optimale, on utilise ici juste les relations $x\%y = x - x//y$ et $x = x//y + x\%y$.)

Ceci conclut la preuve d'hérédité.

Fin de la preuve de correction : on utilise la dernière assertion $\mathcal{A}(K)$, qui entraîne que $r_K$, la valeur retournée, est égale à $b_0 \times (a_0\%2^K)$. Or, par définition de la dernière itération, $a_K = 0$ c'est-à-dire $a_0//2^K = 0$ ce qui signifie que $a_0 = a_0\%2^K$. Donc on a bien $r_K = b_0 a_0$.

Note : une rédaction à l'informaticienne consisterait sans doute à écrire que `ab+resultat` est un invariant de boucle. Pour le démontrer, il faudrait de toute façon écrire quelque chose de relativement semblable à ce qui est écrit plus haut.

## Exercice 3 : multiplication russe en récursif

```
def f(a, b):
    if a == 0:
        return 0

    if a % 2 == 1:
        return b + f(a // 2, b * 2)
    else:
        return f(a // 2, b * 2)
```

Preuve de terminaison et correction : Pour $n \in \mathbb{N}$, notons $\mathcal{A}(n)$ l'assertion « pour tout $m \in \mathbb{N}$, l'appel à $f(n, m)$ termine et renvoie $mn$. »

Montrons $\forall n \in \mathbb{N}, \mathcal{A}(n)$ par récurrence forte.

- Initialisation ok, retour immédiat de la valeur 0.
- Soit $n \in \mathbb{N}^*$. Supposons $\forall k < n, \mathcal{A}(k)$. Montrons $\mathcal{A}(n)$. Comme $n > 0$, par hypothèse de récurrence forte, l'appel à $f(a//2, 2b)$ termine et renvoie $b \times (2(a//2))$. Les instructions conditionnelles peuvent se résumer en disant que la fonction renvoie

  $$b \times (a\%2) + f(a//2, 2b) = b \times (a\%2) + b \times (2(a//2)) = b \times (a\%2 + 2 \times (a//2)) = ba,$$

  ce qu'il fallait démontrer.

## Exercice 4 : PGCD

Montrons que la fonction renvoie le pgcd des deux entrées.

Remarque : la précision « non tous deux nuls » n'est là que que pour éviter un éventuel blocage sur ce point. La « bonne » définition de pgcd fonctionne avec $a$ et $b$ entiers relatifs quelconques et donne $pgcd(0, 0) = 0$. C'est le seul cas où un pgcd est nul.

Terminaison : tant que la boucle tourne, les valeurs de la variable `b` forment une suite strictement décroissante d'entiers naturels. Il n'y a donc qu'un nombre fini de tours de boucle. On note $K$ le nombre de tours de boucle.

Correction : pour $k \in [\![0, K]\!]$, on note $a_k$ et $b_k$ les valeurs des variables `a` et `b` à la fin du $k$-ème tour de boucle (ou avant la boucle pour $k = 0$). On note $\mathcal{A}(k)$ l'assertion « $pgcd(a_0, b_0) = pgcd(a_k, b_k)$ ».

Montrons : $\forall k \in [\![0, K]\!], \mathcal{A}(k)$ par récurrence.

- Initialisation : rien à faire.
- Soit $k \in [\![0, K-1]\!]$. Supposons $\mathcal{A}(k)$. Montrons $\mathcal{A}(k+1)$. Comme $k < K$, on a $b_k > 0$ ce qui autorise la division euclidienne par $b_k$. L'instruction dans la boucle donne $a_{k+1} = b_k$ et $b_{k+1} = a_k \% b_k$. Or, la division euclidienne de $a_k$ par $b_k \neq 0$ s'écrit $a_k = b_k q + a_k \% b_k$. On en déduit, avec les propriétés du pgcd, que

$$pgcd(a_{k+1}, b_{k+1}) = pgcd(b_k, a_k \% b_k) = pgcd(b_k, a_k - b_k q) = pgcd(b_k, a_k)$$

  En utilisant l'hypothèse de récurrence, on en déduit $pgcd(a_0, b_0) = pgcd(a_{k+1}, b_{k+1})$. Ceci conclut l'hérédité.

La dernière assertion $\mathcal{A}(K)$ donne $pgcd(a_0, b_0) = pgcd(a_K, b_K) = pgcd(a_K, 0)$ puisque $b_K = 0$ par définition de $K$. Enfin, $pgcd(a_K, 0) = a_K$, qui est bien la valeur renvoyée.

**Version récursive**, toujours avec a et b naturels non tous deux nuls.

```
def f(a, b):
    if b == 0:
        return a
    return f(b, a % b)
```

Terminaison et correction : Récurrence forte sur la deuxième variable (qui décroit strictement lors des appels successifs).

Pour $n \in \mathbb{N}$, notons $\mathcal{A}(n)$ l'assertion « pour tout $m \in \mathbb{N}$, l'appel à $f(m, n)$ termine et renvoie le pgcd de $m$ et $n$. ».

- L'assertion $\mathcal{A}(0)$ est vraie : pour tout $m$, on renvoie $m$, qui est bien le pgcd de $m$ et de zéro.
- Soit $n \in \mathbb{N}^*$. Supposons $\mathcal{A}(k)$ pour tout naturel $k < n$. Montrons $\mathcal{A}(n)$. Soit donc $m$ un naturel. Par définition de l'opérateur %, on a $m\%n < n$. Donc $f(n, m\%n)$ termine et renvoie le pgcd de $n$ et $m\%n$. Comme $n$ est non nul, on peut écrire la division euclidienne $m = nq + m\%n$. On a alors

$$pgcd(m, n) = pgcd(nq + m\%n, n) = pgcd(m\%n, n)$$

## Exercice 5 : recherche dichotomique

Preuve de terminaison et correction.

On va montrer que pour tout $x \in \mathbb{R}$ et toute liste $L$ triée par ordre croissant, l'appel à $f(x, L)$ termine et renvoie vrai si $x$ est dans $L$, et faux sinon.

Soit $x \in \mathbb{R}$. On fait maintenant une récurrence forte sur la longueur de la liste. Pour $n \in \mathbb{N}$, notons $\mathcal{A}(n)$ l'assertion « pour toute liste $L$ triée par ordre croissant et de longueur $n$, l'appel à $f(x, L)$ termine et renvoie Vrai si $x$ est dans $L$, et Faux sinon. »

- L'assertion $\mathcal{A}(0)$ est vraie.
- Soit $n \in \mathbb{N}^*$ Supposons que pour tout naturel $k < n$, $\mathcal{A}(k)$. Montrons $\mathcal{A}(n)$. Soit donc $L$ une liste de longueur $n$. La liste n'est pas vide car $n > 0$, et on a donc $0 \leq p = n//2 < n$, $p$ est un indice valide de la liste $L$ et les listes `L[p+1:]` et `L[:p]` sont des sous-listes de $L$ (éventuellement vides) correspondant aux indices strictement supérieurs ou strictement inférieurs à $p$. Elles sont donc de longueur strictement plus petite. Par hypothèse de récurrence forte, les appels récursifs sur ces sous-listes terminent et renvoient le booléen d'appartenance de $x$ à la sous-liste en question. Passons en revue les trois cas possibles pour la comparaison entre $x$ et $L[p]$ :

— S'il y a égalité, on renvoie Vrai, ce qui correspond bien au résultat souhaité.
— Si $L[p] < x$, comme $L$ est triée par ordre croissant, $x$ est dans $L$ ssi $x$ est dans la sous-liste $L[p+1:]$ correspondante aux indices $> p$. Par hypothèse de récurrence forte, la fonction renvoie le bon booléen.
— Si $L[p] > x$, là aussi comme $L$ est triée par ordre croissant, $x$ est dans $L$ ssi $x$ est dans la sous-liste $L[:p]$ correspondante aux indices $< p$. Par hypothèse de récurrence forte, la fonction renvoie le bon booléen.

Ceci conclut l'hérédité sous hypothèse de récurrence forte.

## Exercice 6 : recherche de sous-listes

Note : on suppose toujours que $n \in \mathbb{N}$ à chaque fois que la fonction est appelée.

```
def estSousListePosition(L,M,n):
  if n + len(L) > len(M) : return False
  for k in range(len(L)):
    if L[k] != M[n+k]: return False
  return True

def estSousListe(L,M):
  for n in range(len(M)):
    if estSousListePosition(L,M,n): return True
  return False
```

**Preuve de terminaison et correction pour la fonction auxiliaire** : La boucle for fait un nombre fini d'itérations, la fonction termine donc.

La première instruction conditionnelle renvoie Faux si $L$ ne peut pas être une sous-liste de $M$ en position $n$ pour des raisons de taille et d'indice.

Dans le cas contraire, c'est que pour tout $k \in [\![0, long(L)-1]\!]$, le terme $M[n+k]$ est bien défini.

Si on effectue la boucle en entier sans renvoyer Faux, c'est que pour tout $k$, on avait $M[n+k] = L[k]$, autrement dit $L$ est une sous-liste de $M$ en position $n$. Dans ce cas on renvoie bien Vrai.

Dans le cas contraire, si on renvoie Faux en cours de boucle, c'est qu'une égalité était fausse, donc que $L$ n'était pas une sous-liste de $M$ en position $n$.

**Preuve de terminaison et correction pour la fonction principale** :

La fonction termine car une boucle for termine.

La fonction renvoie bien Vrai s'il existe $n \in [\![0, long(M)-1]\!]$ tel que $L$ soit une sous-liste de $M$ en position $n$. Elle renvoie Faux sinon. C'est bien ce qu'il fallait montrer.